GLOBECOM 2024 - 2024 IEEE Global Communications Conference | 979-8-3503-5125-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/GLOBECOM52923.2024.10900982

BPVFL: A Bidirectional Privacy-Preserving Verifiable Federated Learning Framework with Homomorphic Encryption

Jingwei Liu
Shaanxi Key Laboratory of Blockchain and Secure Computing
Xidian University
Xi'an, 710071, China
jwliu@mail.xidian.edu.cn

Sijing Chen
Shaanxi Key Laboratory of Blockchain and Secure Computing
Xidian University
Xi'an, 710071, China
sjc@stu.xidian.edu.cn

Junrong Zhu
Shaanxi Key Laboratory of Blockchain and Secure Computing
Xidian University
Xi'an, 710071, China
jrzhu@stu.xidian.edu.cn

Rong Sun
State Key Laboratory of ISN
Xidian University
Xi'an, 710071, China
rsun@mail.xidian.edu.cn

Xiaojiang Du
Department of Electrical and Computer Engineering
Stevens Institute of Technology
Hoboken, USA
dxj@ieee.org

Mohsen Guizani
Department of Machine Learning
Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI)
Abu Dhabi, 999041, UAE
mguizani@ieee.org

***Abstract*—Federated learning, while advancing data privacy, faces risks of sensitive information leakage through parameter updates, making it susceptible to inference and data reconstruction attacks. Fraudulent behaviors by central servers or clients can undermine the integrity of model training, thereby reducing accuracy and affecting decision-making quality. This paper introduces a bidirectional, privacy-preserving verifiable federated learning framework(BPVFL) built on homomorphic encryption and a novel three-party zero-knowledge protocol. This framework guarantees the integrity of server aggregation and the credibility of the information uploaded by clients. Experimental results demonstrate that BPVFL effectively protects client privacy, prevents fraud by servers and certain clients, and efficiently handles numerous client disconnections with minimal overhead.**

***Index Terms*—Federated learning, Privacy preservation, Homomorphic encryption, Zero-knowledge.**

I. Introduction

Federated learning has surfaced as a promising distributed computational framework in the era of big data. It enables collaborative training of models across multiple devices, all while ensuring the privacy of training datasets [1]–[4]. However, federated learning grapples with several challenges, including risks of privacy leakage [5] and deception from dishonest participants [6], [7].

Current federated learning frameworks that ensure both privacy protection and verifiability primarily rely on secure aggregation and homomorphic hash functions. VerifyNet [6] incorporates a dual-masking protocol utilizing secure multi-party computation to safeguard the privacy of local model parameters and employs a homomorphic hash function to ascertain the integrity of the cloud server's parameter aggregation outcomes. VeriFL [8] reduces both computation and communication burdens during the verification phase compared to VerifyNet, while still maintaining client privacy. However, these frameworks overlook the potential dishonesty of clients in practical settings [9] and limit the maximum number of tolerable dropouts based on the secret sharing threshold.

PVFL [10], built on differential privacy, introduces a lightweight, privacy-preserving federated learning framework with integrated verification capabilities. However, the use of differential privacy may compromise model accuracy. Additionally, the framework does not account for dishonest clients, limiting its effectiveness and practicality in real-world applications.

To address these limitations and enable mutual authentication between servers and clients, we propose a new privacy-preserving and mutually verifiable federated learning framework, named BPVFL. BPVFL is based on homomorphic encryption, homomorphic hash, and zero-knowledge proofs. The framework supports up to $N-1$ client dropouts (with N clients in total), and eliminates the need for clients to negotiate keys or share data pairwise, thereby reducing communication and computation overheads.

In summary, the main contributions of this paper are:

- We propose an efficient bidirectional privacy-preserving verifiable federated learning framework that allows verification of the consistency between the server's aggregation results and the encrypted model parameters uploaded by clients, without revealing private data.
- The framework employs a sliding window sampling technique, which significantly reduces computation and communication overhead during the verification process.

Authorized licensed use limited to: Anhui University. Downloaded on April 23,2026 at 10:42:46 UTC from IEEE Xplore. Restrictions apply.

- The scheme offers fault tolerance: even with up to m client disconnections during training or validation (where there are N clients and $0 \leq m \leq N-1$), the model aggregation and bidirectional validation process can still be maintained.
- The scheme eliminates the need for a fully trusted third party, enhancing the system's independence and security.

The remainder of the paper is structured as follows: Section II covers the preliminaries. Section III elaborates on the proposed framework. Section IV discusses the security and verifiability analyses. Section V evaluates the performance of BPVFL. Section VI concludes the paper.

II. Preliminaries

In this section, we provide a brief overview of the preliminaries involved in our work, including federated learning and linear homomorphic hashing.

A. Federated Learning

Federated Learning (FL) is a decentralized approach to machine learning that allows for model training across distributed data repositories without the need to centralize the data [3]. The process involves distributing a global model across a network of participating clients, each of which contributes to the training using its local data. The process begins with a shared model, initialized by a centralized server. Clients download this model and conduct local training on their respective datasets. Afterward, they send model updates, typically in the form of gradients, back to the server. The server aggregates these updates to refine the global model, which is then redistributed to the clients for further iterations. This iterative process continues until the model meets the convergence criteria.

B. Linear Homomorphic Hash

A linear homomorphic hash function operates through the following three polynomial-time algorithms:

- $LHH.Gen(1^k)$: This algorithm takes a security parameter k as input and outputs public parameters, including the description of a cyclic group $\mathbb{G}$ of prime order q, a prime number o, and a generator $g \in \mathbb{G}$. To streamline subsequent processes, these public parameters are also used as the initial parameters in both $LHH.Hash$ and $LHH.Eval$ functions.
- $LHH.Hash(x)$: The algorithm processes an input in the form of a d-dimensional vector x and produces a linearly homomorphic hash: $H(x) = g^x \mod o$, which is computed as $(g^{x_1} \mod o, \ldots, g^{x_d} \mod o)$.
- $LHH.Eval(a_1, \ldots, a_l, h_1, \ldots, h_l)$: The algorithm computes the linear combination of l hashes and the corresponding l coefficients to produce the resultant hash $H*$.

III. Framework

In this section, we present the concrete structure of our framework and delve into its technical specifics. As illustrated in Fig. 1, the BPVFL framework involves three types of participants:

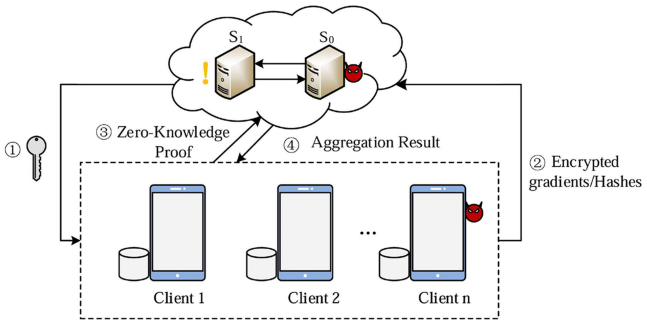


Fig. 1. The structure of BPVEL

1) Aggregation cloud server (S_0): The aggregation cloud server is dishonest. It may falsify aggregation results to disrupt the training process and attempt to infer private data by accessing clients' model parameters.

2) Verification cloud server (S_1): The verification cloud server is semi-honest. Although it may be curious about the private data of other clients, it strictly adheres to protocols such as key generation, distribution, and proof verification.

3) N Clients: Among the N clients, up to m may behave maliciously $(0 \leq m \leq N-1)$, while the rest are semi-honest. Malicious clients may fabricate verification information to obstruct the verification process, whereas semi-honest clients follow the protocols as required. Additionally, each client may be interested in deducing private information from the model parameters of others.

Algorithm 1 Federated Learning in the k-th Iteration

Participants: Server S_0, Server S_1, Clients $C_1, C_2, ..., C_n$
Output: Aggregated global model parameters M_{global} and hash value $H(M_{global})$
Phase 1: Encryption of Input
S_1 generates public/private keys (pk, sk) and broadcasts pk.
S_1 generates the hash parameters (g, o) and distributes them to each client.
for each client C_i **do**
    Train the local model M_i;
    Encrypt $Enc_{pk}(M_i)$ and compute hash value $H(M_i)$;
    Upload $Enc_{pk}(M_i)$ and $H(M_i) \rightarrow S_0$.
end for
Phase 2: Efficient Communication and Computation TP-ZKP
S_0 sends a window W of length k and a $k \times k$ permutation matrix $A \rightarrow C_i$.
for each client C_i **do**
    S_0, S_1 and C_i carry out TP-ZKP using the extracted parts of $Enc_{pk}(M_i)$, and $H(M_i)$ with W.
end for
Phase 3: Aggregation
S_0 computes the aggregate value $Enc_{pk}(M_{global})$ and $H(M_{global})$;
S_1 decrypts $Enc_{pk}(M_{global})$ to M_{global} and completes the integrity verification.

Authorized licensed use limited to: Anhui University. Downloaded on April 23,2026 at 10:42:46 UTC from IEEE Xplore. Restrictions apply.

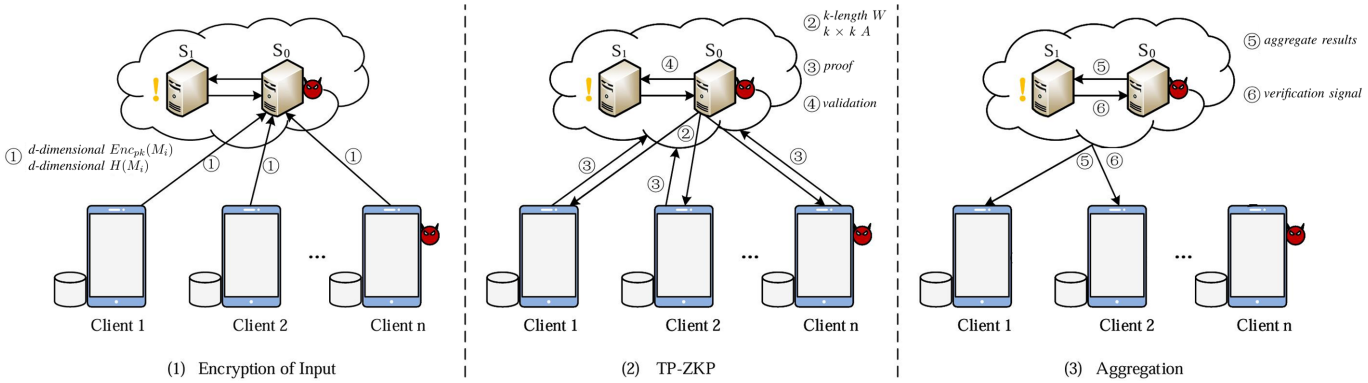


Fig. 2. Overview of the FL framework

As depicted in Fig. 2, we categorize a single iteration of FL process into three distinct phases: 1) Encryption of Input, 2) Efficient three-party zero-knowledge protocol (TP-ZKP), and 3) Aggregation. Algorithm 1 outlines the general workflow during the k-th iteration of the training process.

In Phase 1, S_1 generates a public-private key pair (pk, sk), as described below:

$$(pk, sk) \leftarrow KenGen() \tag{1}$$

It broadcasts pk to all participating entities while retaining sk for itself. Subsequently, S_1 randomly selects a generator g from a cyclic group $\mathbb{G}$ and a large prime number o as parameters for the homomorphic hash function. S_1 then conveys g and o to all clients through secure channels. Let the set of clients that received pk, g and o be denoted by U.

Next, client C_i engages in local training on its dataset D_i to derive a d-dimensional local model parameter M_i. Subsequently, it computes the hash value $H(w_i)$, which is expressed as follows:

$$\begin{aligned} H(M_i) &= g^{M_i} \bmod o \\ &= (g^{M_i^1} \bmod o, g^{M_i^2} \bmod o, ..., g^{M_i^d} \bmod o) \end{aligned} \tag{2}$$

C_i encrypts the model parameter using the public key to calculate $Enc_{pk}(M_i)$ and transmits both $Enc_{pk}(M_i)$ and $H(M_i)$ to S_0 via a secure channel. The encrypted parameter $Enc_{pk}(M_i)$ is structured as follows:

$$Enc_{pk}(M_i) = (Enc_{pk}(M_i^1), ..., Enc_{pk}(M_i^d)) \tag{3}$$

During Phase 2, upon receiving the data uploaded by a subset of users $U_1 \subset U$, S_0 generates a randomized $k \times k$ permutation matrix A and a window W of length k, which are then conveyed to client C_i. In response, C_i selects a k-dimensional random vector r, encrypts it with pk to send to S_0, and generates a subsequent proof to send to S_1:

$$M_i^k = (M_i^{W_1}, M_i^{W_2}, ..., M_i^{W_k}) \tag{4}$$

$$M_i{}' = M_i^k A + r \tag{5}$$

$$\textit{Proof}\ \pi = (M_i{}', Enc_{pk}(r)) \rightarrow S_1 \tag{6}$$

After receiving $Enc_{pk}(r)$ from C_i, S_0 calculates:

$$Enc_{pk}(M_i)^k = (Enc_{pk}(M_i^{W_1}), ..., Enc_{pk}(M_i^{W_k})) \tag{7}$$

$$H(M_i)^k = (g^{M_i^{W_1}} \bmod o, ..., g^{M_i^{W_k}} \bmod o) \tag{8}$$

$$Enc_{pk}(M_i)' = Enc_{pk}(M_i)^k A + Enc_{pk}(r) \rightarrow \text{Broadcast} \tag{9}$$

$$H(M_i)' = H(M_i)^k A \rightarrow \text{Broadcast} \tag{10}$$

Then, S_1 verifies:

$$Dec_{sk}(Enc_{pk}(M_i)') \xrightarrow{?} M_i{}' \tag{11}$$

$$H(M_i{}')/H(Dec_{sk}(Enc_{pk}(r))) \xrightarrow{?} H(M_i)' \tag{12}$$

Suppose that within the specified time interval t, S_1 receives proofs from $|U_2|$ clients, where the set of these clients is denoted as $U_2 \subset U_1$. If the validation is successful, S_1 sends a confirmation message "True" to S_0; Otherwise, it sends "False".

In Phase 3, once a sufficient number of "True" responses have been accumulated, S_0 proceeds to aggregate the encrypted model parameters and hash values from the validated clients, then disseminates the aggregated results to them. We denote the set of validated clients in this phase as $U_3 \subset U_2$. Assuming that each client holds an equal amount of data, the resulting aggregated parameters and hash value can be expressed as follows:

$$\begin{aligned} Enc_{pk}(M_{global}) = (&\prod\nolimits_{i \in U_3} Enc_{pk}(M_i^1)^{1/|U_3|}, \\ &..., \prod\nolimits_{i \in U_3} Enc_{pk}(M_i^d)^{1/|U_3|}) \end{aligned} \tag{13}$$

$$\begin{aligned} H(M_{global}) = (&\prod\nolimits_{i \in U_3} (g^{M_i^1} \bmod o)^{1/|U_3|}, \\ &..., \prod\nolimits_{i \in U_3} (g^{M_i^d} \bmod o)^{1/|U_3|}) \end{aligned} \tag{14}$$

Finally, S_0 sends the aggregated hash value and the aggregated encrypted model parameters to S_1. S_1 then decrypts the aggregated ciphertexts of model parameters and performs an integrity verification. If the decrypted model parameters correspond with the homomorphic hash value, S_1 broadcasts

Authorized licensed use limited to: Anhui University. Downloaded on April 23,2026 at 10:42:46 UTC from IEEE Xplore. Restrictions apply.

a signal of successful verification, notifying all participating parties to proceed with the new global model parameters for the next training iteration. Conversely, if a discrepancy is detected, the training process will be halted. The integrity verification of the server's aggregated results can be expressed as follows:

$$H(Dec_{sk}(Enc_{pk}(M_{global}))) \xrightarrow{?} H(M_{global}) \tag{15}$$

IV. Analysis

In this section, a thorough analysis of the proposed framework is carried out, focusing on two key aspects: *Security* and *Verifiability*.

A. Security analysis

1) Confidentiality:

Theorem 1. *S_0 cannot obtain the plaintext of $Enc_{pk}(M_i)$ solely with the public key pk.*

Proof. In BPVFL, Paillier homomorphic encryption is employed to protect the privacy of client-uploaded gradients. The computational intractability of the discrete logarithm problem [11] ensures that Paillier encryption remains secure against decryption attempts by adversaries. □

2) Unforgeability:

Theorem 2. *In the aggregation phase, S_0 cannot forge a valid hash value to pass the participants' verification.*

Proof. S_0 may attempt to use a poisoning model as the aggregation result and send it to each client to undermine the model's accuracy or mislead the training process. To resist this malicious behavior, the scheme requires that the hash value aggregated by S_0 meet the following condition:

$$H(M_{global}) = g^{M_{global}} \bmod o \tag{16}$$

However, S_0 cannot obtain g that is a large prime number randomly chosen by S_1 and all clients. The probability that S_0 successfully forges the corresponding hash value is:

$$\Pr(\text{Successful forgery}) = \prod_{i\in[d]} 2^{-k} \leq \text{negl}(k) \tag{17}$$

Therefore, S_0 cannot forge a valid homomorphic hash value to pass the participants' verification. □

B. Verifiability analysis

Theorem 3. *S_1 possesses the capability to authenticate the integrity of the aggregation results from S_0 while maintaining the secrecy of the clients' model parameters.*

Proof. By Theorem 2, the unforgeability of the homomorphic hash values generated during the aggregation phase ensures the integrity of the process. Assuming no collusion between the servers, S_1 can verify if S_0 has executed the process faithfully. □

Theorem 4. *S_0 can verify the correlation between the clients' model parameters and their hash values without compromising the confidentiality of the parameters.*

Proof. In one iteration, based on the data uploaded by the clients, S_0 can calculate the linear transformation results of the model parameters and homomorphic hash values and send them to S_1:

$$\begin{cases} (Enc_{pk}({M_i}^{W_1}), ..., Enc_{pk}({M_i}^{W_k}))A \cdot \\ (Enc_{pk}(r_1), ..., Enc_{pk}(r_k))^{\mathsf{T}} \\ (H(M_i^{W_1}), ..., H(M_i^{W_k}))A \end{cases} \tag{18}$$

Meanwhile, S_1 receives the following from the clients:

$$\begin{cases} ({M_i}^{W_1}, ..., {M_i}^{W_k})A + (r_{W_1}, ..., r_{W_k}) \\ (Enc_{pk}(r_{W_1}), ..., Enc_{pk}(r_{W_k})) \end{cases} \tag{19}$$

Based on the above, S_1 can ascertain the following:

- Do the model parameters previously uploaded by clients correspond to the plaintext resulting from linear transformations in the subsequent proofs?
- Do the model parameters initially uploaded by clients correspond to the homomorphic hash values?

Without access to the specifics of the permutation matrix A (generated by S_1) and the random vector r (generated by S_0), and in the absence of collusion among the three participating entities, the privacy of the clients' uploaded model parameters is safeguarded. □

V. Performance Evaluation

This section presents the implementation of BPVFL, using VerifyNet [6] and VeriFL [8] as the baselines. We evaluate both communication and computation overheads.

A. Complexity Analysis

1) Communication Overhead: In [6], VerifyNet employs zero-knowledge proofs to maintain the integrity of aggregated results, where the size of the proof corresponds to the number of model parameters. In [8], VeriFL accomplishes verification via a fixed-length hash value. Nevertheless, in both [6] and [8], clients are required to exchange secret shares to mask the gradients and accommodate scenarios involving offline clients, resulting in a communication overhead that corresponds to the count of offline clients, denoted as Φ.

The proposed framework adopts Paillier encryption to secure model parameters, thereby eliminating the need for secret share exchanges among clients. Additionally, this scheme innovatively samples k model parameters, resulting in a communication overhead for client verification that is linearly related to k, significantly reducing the communication burden. Let d be the dimension of the model parameters, n the number of clients, and h the communication expansion factor induced by the homomorphic encryption algorithm. The communication costs of these frameworks are shown in Table I and Table II.

2) Computation Overhead: As shown in Table III, both [6] and [8] require each client to negotiate with other clients to generate shared keys and secret shares to obfuscate their gradients, resulting in a computation overhead of $O(dn)$. The server must retrieve the shared keys from online clients and the secret shares from offline clients, incurring a computation overhead of $O(dn)$. In BPVFL, clients encrypt their model

Authorized licensed use limited to: Anhui University. Downloaded on April 23,2026 at 10:42:46 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMMUNICATION OVERHEAD

	Clients	Server/Servers
[6]	$O(hd)$	$O(hdn)$
[8]	$O(dn)$	$O(dn^2)$
BPVFL	$O(h(d+k))$	$O(hn(d+k))$

TABLE II
COMMUNICATION OVERHEAD FOR VERIFICATION

	Each client	Server/Servers
[6]	$O(d\Phi)$	$O(dn-\Phi)$
[8]	$O(\Phi)$	$O(n-\Phi)$
BPVFL	$O(d+k)$	$O(dk-\Phi)$

parameters, leading to a computation overhead of $O(Hd)$, where H denotes the computational cost associated with homomorphic encryption. The server then aggregates these encrypted parameters from all clients, resulting in a computation overhead of $O(Hdn)$.

During the verification phase, both [6] and [8] incur encryption and decryption cost of $O(d)$ for the aggregated results. Additionally, they require extra secret-sharing computations of $O(N)$ in the aggregation phase, where N represents the number of offline clients. In BPVFL, the computational cost for each client to randomly select and perform linear transformations on k parameters is $O(k)$. Combined with the $O(d)$ cost for aggregating the parameters, the total verification computational cost is $O(k+d)$.

B. Experimental Evaluation

We evaluate BPVFL on the MNIST dataset [12] and conducte the simulation on a 64-bit Ubuntu 20.04 LTS cloud server equipped with one NVIDIA A16 GPU, three Intel Xeon Gold 5318Y CPUs (2.10GHz), and 14GB RAM. Paillier encryption and decryption were implemented using the Python Paillier Library.

TABLE III
COMPUTATION OVERHEAD

	Each client	Server/Servers
[6]	$O(dn)$	$O(dn)$
[8]	$O(dn)$	$O(dn)$
BPVFL	$O(Hd)$	$O(Hdn)$

TABLE IV
COMPUTATION OVERHEAD FOR VERIFICATION

	Computation Overhead
[6]	$O(d+N)$
[8]	$O(d+N)$
BPVFL	$O(d+k)$

TABLE V
COMPUTATION OVERHEAD FOR VERIFICATION

Dropout	Framework	Role	Computation overhead
10%	VerifyNet	Client	4.4×10^5 ms
		Server	9.7×10^5 ms
	VeriFL	Client	4.0×10^3 ms
		Server	7.1×10^3 ms
	BPVFL	Client	1.6×10^3 ms
		Server	3.9×10^5 ms
30%	VerifyNet	Client	4.2×10^5 ms
		Server	1.8×10^6 ms
	VeriFL	Client	3.9×10^3 ms
		Server	2.1×10^5 ms
	BPVFL	Client	1.6×10^3 ms
		Server	3.1×10^5 ms

1) Dimension-related Communication Overhead: We set the total number of clients as $N=500$ and define the threshold $\tau=N/2$. We then explored scenarios with offline client ratios of $0.1N$ and $0.3N$. Fig. 3 illustrates the communication overhead incurred by each framework during a single iteration.

As depicted in Fig. $3(a)$ and $3(b)$, BPVFL exhibits superior performance over [6] and [8] concerning the communication overhead for low-dimensional parameters. However, as the dimension increases, it shows slight inferiority compared to [8] yet maintains an advantage over [6]. This is because, in [8], the verification values have a fixed length, whereas in [6] and BPVFL, the length of the verification values is correlated with the dimension d.

As illustrated in Fig. $3(c)$ and $3(d)$, the overall communication overhead for clients within BPVFL ranks higher than that of [8], but remains lower than [6]. Conversely, the total communication overhead for servers is more favorable compared to the other two methods.

2) Dropout-Independence Communication Overhead: We set the dimension d at 1000, set the number of clients N at 500, and assigned $\tau=250$. As shown in Fig. 3, the communication overhead per client's verification in BPVFL is fixed, and the server's communication overhead diminishes linearly with Φ. In contrast, in [8], the client-side communication overhead increases linearly with Φ. [6] has the largest communication overhead. When $\Phi > N/2$, both [8] and [6] will halt the training, whereas BPVFL can accommodate up to $N-1$ offline clients.

3) Computation Overhead: With the parameters set to $d=10000$, $N=500$, $\tau=250$, and considering a single batch, we analyzed the verification computational overhead of BPVFL, as presented in Table V.

In both [6] and [8], the computational overhead increases as the number of dropouts rises. In contrast, BPVFL renders its computational overhead independent of Φ.

Authorized licensed use limited to: Anhui University. Downloaded on April 23,2026 at 10:42:46 UTC from IEEE Xplore. Restrictions apply.

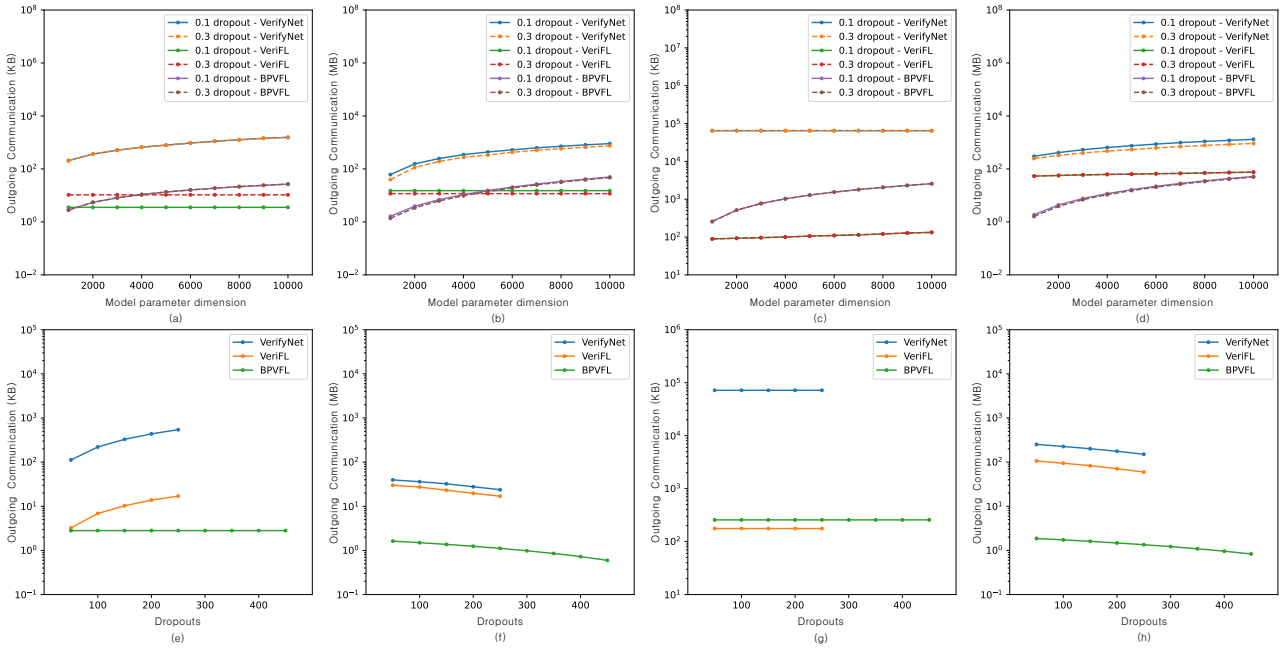


Fig. 3. Comparisons of outgoing communication for each framework concerning verification and overall overhead, considering different vector dimensions and numbers of dropouts. Figures (a) through (d) detail the communication overhead across varying vector dimensions, while (e) through (h) emphasize the overhead associated with different counts of dropouts. Specifically, (a) and (e) demonstrate the per-client outgoing communication for verification at each iteration; (b) and (f) show the central server's (CS) outgoing communication for verification at each iteration; (c) and (g) depict the total per-client outgoing communication at each iteration; (d) and (h) reveal the total outgoing communication of the CS at each iteration.

VI. Conclusion

This paper introduced a novel Bidirectional Privacy-Preserving Verifiable Federated Learning framework (BPVFL). The framework leverages homomorphic encryption techniques to protect privacy of client-side model parameters, employs a unique three-party zero-knowledge protocol to enable bidirectional authentication between the server and clients, and adopted a window sampling method to reduce computational and communication overheads. Experimental results demonstrate that BPVFL outperforms existing solutions in terms of communication and computation overheads during the verification phase and significantly enhances robustness against client dropouts. Future endeavors could concentrate on defending against malicious adversaries executing model poisoning or data poisoning attacks, thereby broadening the applicability of our framework to a wider range of federated learning scenarios.

Acknowledgment

This work is supported by the Key Research and Development Program of Shaanxi (No. 2023-ZDLGY-34, No. 2021ZDLGY05-03, No. 2020ZDLGY05-04).

References

[1] L. Wu, X. Du, M. Guizani, and A. Mohamed, "Access control schemes for implantable medical devices: A survey," *IEEE Internet of Things Journal*, vol. 4, no. 5, pp. 1272–1283, 2017.

[2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.

[3] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao, "A survey on federated learning," *Knowledge-Based Systems*, vol. 216, p. 106775, 2021.

[4] X. Du, Y. Xiao, and F. Dai, "Increasing network lifetime by balancing node energy consumption in heterogeneous sensor networks: Research articles," *Wirel. Commun. Mob. Comput.*, vol. 8, no. 1, p. 125–136, 2008.

[5] Q. Li, Z. Wen, Z. Wu, S. Hu, N. Wang, Y. Li, X. Liu, and B. He, "A survey on federated learning systems: Vision, hype and reality for data privacy and protection," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 4, pp. 3347–3366, 2023.

[6] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "Verifynet: Secure and verifiable federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 911–926, 2020.

[7] X. Du, M. Guizani, Y. Xiao, and H.-H. Chen, "Defending dos attacks on broadcast authentication in wireless sensor networks," in Proc. of IEEE International Conference on Communications, 2008, pp. 1653–1657.

[8] X. Guo, Z. Liu, J. Li, J. Gao, B. Hou, C. Dong, and T. Baker, "Verifl: Communication-efficient and fast verifiable aggregation for federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1736–1751, 2021.

[9] J. Brown and X. Du, "Detection of selective forwarding attacks in heterogeneous sensor networks," in Proc. of IEEE International Conference on Communications, 2008, pp. 1583–1587.

[10] H. Zhou, G. Yang, Y. Huang, H. Dai, and Y. Xiang, "Privacy-preserving and verifiable federated learning framework for edge computing," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 565–580, 2023.

[11] A. Acar, H. Aksu, A. S. Uluagac, and M. Conti, "A survey on homomorphic encryption schemes: Theory and implementation," *ACM Computing Surveys (Csur)*, vol. 51, no. 4, pp. 1–35, 2018.

[12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

Authorized licensed use limited to: Anhui University. Downloaded on April 23,2026 at 10:42:46 UTC from IEEE Xplore. Restrictions apply.